

Reimagining Documentation as Code with Advanced Python Systems

"""

```
from __future__ import annotations
```

```
import abc
```

```
import asyncio
```

```
import concurrent.futures
```

```
import dataclasses
```

```
import functools
```

```
import hashlib
```

```
import inspect
```

```
import json
```

```
import math
```

```
import os
```

```
import sys
```

```
import time
```

```
from collections import defaultdict, deque
```

```
from contextlib import asynccontextmanager, contextmanager
```

```
from dataclasses import dataclass
```

```
from datetime import datetime, timedelta
```

```
from typing import (
```

```
    Any,
```

```
    AsyncGenerator,  
    Callable,  
    Deque,  
    Dict,  
    Generator,  
    Generic,  
    Iterable,  
    List,  
    Optional,  
    Protocol,  
    Sequence,  
    Tuple,  
    Type,  
    TypeVar,  
    Union,  
)
```

```
# Optional: third-party-like lightweight validation using dataclasses for portability
```

```
T = TypeVar("T")
```

```
# -----
```

```
# Content Models (Data Layer)
```

```
# -----
```

```
@dataclass(frozen=True)
```

```
class ContentNode:
```

```
    """Represents a minimal content unit for a knowledge base.
```

```
    Includes semantic metadata for Knowledge Graph integration and embeddings.
```

```
    """
```

```
    id: str
```

```
    title: str
```

```
    body: str
```

```
    created_at: datetime
```

```
    updated_at: datetime
```

```
    tags: Tuple[str, ...] = ()
```

```
    locale: str = "en-US"
```

```
    version: int = 1
```

```
    def fingerprint(self) -> str:
```

```
        """Stable content fingerprint used for de-dup and cache keys."""
```

```
        s = f"{self.title}|{self.body}|{self.locale}|{self.version}"
```

```
        return hashlib.sha256(s.encode("utf-8")).hexdigest()
```

```
@dataclass
```

```
class DocumentBundle:
```

```
    """A bundle of content nodes that represent a single doc or topic.
```

Useful for single-source publishing and modular topic-based authoring.

"""

bundle_id: str

nodes: List[ContentNode]

metadata: Dict[str, Any]

Plugin Parser Architecture

class ParserPlugin(Protocol):

"""Parser plugin Protocol for parsing a raw source into ContentNodes.

Advanced recruiters look for plugin architectures and extensibility.

"""

async def parse(self, source: str, **kwargs: Any) -> List[ContentNode]:

...

Simple registry

PARSERS: Dict[str, Type[ParserPlugin]] = {}

```
def register_parser(name: str):
    def _decorator(cls: Type[ParserPlugin]):
        PARSERS[name] = cls
        return cls

    return _decorator
```

```
@register_parser("markdown")
```

```
class MarkdownParser:
```

```
    """A fast markdown -> ContentNode parser.
```

Includes advanced features: front-matter extraction, code block indexing, and lightweight AST.

(Note: simplified for sample)

```
    """
```

```
    async def parse(self, source: str, **kwargs: Any) -> List[ContentNode]:
```

```
        nodes: List[ContentNode] = []
```

```
        paragraphs = [p.strip() for p in source.split("\n\n") if p.strip()]
```

```
        now = datetime.utcnow()
```

```
        for i, p in enumerate(paragraphs):
```

```
            title = p.split("\n", 1)[0][:80]
```

```

        node = ContentNode(id=f"md-{hashlib.md5(p.encode()).hexdigest()}", title=title,
body=p, created_at=now, updated_at=now)

        nodes.append(node)

    return nodes

```

```

@register_parser("simple-html")

```

```

class SimpleHTMLParser:

```

```

    async def parse(self, source: str, **kwargs: Any) -> List[ContentNode]:

```

```

        # For the sample, we strip tags naively. In production you'd use a real parser.

```

```

        import re

```

```

        text = re.sub(r"<[^>]+>", " ", source)

```

```

        now = datetime.utcnow()

```

```

        return [ContentNode(id=f"html-{hashlib.md5(text.encode()).hexdigest()}", title=text[:80],
body=text, created_at=now, updated_at=now)]

```

```

# -----

```

```

# Dependency Injection - tiny container

```

```

# -----

```

```

class Container:

```

```

    """A micro dependency-injection container useful for tests and modularity."""

```

```

    def __init__(self):

```

```
self._providers: Dict[str, Callable[["Container"], Any]] = {}
```

```
self._cache: Dict[str, Any] = {}
```

```
def provide(self, name: str, provider: Callable[["Container"], Any]) -> None:
```

```
    self._providers[name] = provider
```

```
def get(self, name: str) -> Any:
```

```
    if name in self._cache:
```

```
        return self._cache[name]
```

```
    if name not in self._providers:
```

```
        raise KeyError(name)
```

```
    val = self._providers[name](self)
```

```
    self._cache[name] = val
```

```
    return val
```

```
container = Container()
```

```
# -----
```

```
# Observability (placeholder for OpenTelemetry)
```

```
# -----
```

```
class Tracer:
```

```
    def start_span(self, name: str):
```

```
        # lightweight context manager placeholder
```

```

@contextmanager
def _span():
    start = time.time()
    print(f"[trace:start] {name}")
    try:
        yield
    finally:
        end = time.time()
        print(f"[trace:end] {name} (duration={end-start:.3f}s)")

return _span()

```

```

container.provide("tracer", lambda c: Tracer())

```

```

# -----

```

```

# Caching & LRU (performance pattern)

```

```

# -----

```

```

@functools.lru_cache(maxsize=256)

```

```

def expensive_text_fingerprint(text: str) -> str:

```

```

    # simulate expensive CPU-bound operation

```

```

    time.sleep(0.01)

```

```

    return hashlib.sha1(text.encode()).hexdigest()

```

```
# -----
```

```
# Rate Limiter - Token Bucket
```

```
# -----
```

```
class TokenBucket:
```

```
    def __init__(self, rate: float, capacity: float):
```

```
        self._rate = rate
```

```
        self._capacity = capacity
```

```
        self._tokens = capacity
```

```
        self._last = time.monotonic()
```

```
    def consume(self, tokens: float = 1.0) -> bool:
```

```
        now = time.monotonic()
```

```
        elapsed = now - self._last
```

```
        self._last = now
```

```
        self._tokens = min(self._capacity, self._tokens + elapsed * self._rate)
```

```
        if self._tokens >= tokens:
```

```
            self._tokens -= tokens
```

```
            return True
```

```
        return False
```

```
# -----
```

```
# Async ingestion pipeline
```

```
# -----
```

```
class Ingestor:
```

```
    """Ingest raw sources into the system using async pipelines, batching, and concurrency."""
```

```
    def __init__(self, parser_name: str, rate_limit: Optional[TokenBucket] = None):
```

```
        if parser_name not in PARSERS:
```

```
            raise ValueError("unknown parser")
```

```
        self._parser_cls = PARSERS[parser_name]
```

```
        self._rate = rate_limit or TokenBucket(rate=100.0, capacity=100.0)
```

```
        self._executor = concurrent.futures.ThreadPoolExecutor(max_workers=4)
```

```
    async def _parse_in_thread(self, source: str) -> List[ContentNode]:
```

```
        loop = asyncio.get_event_loop()
```

```
        parser = self._parser_cls()
```

```
        # call async parse directly, in case plugin does async ops
```

```
        if inspect.iscoroutinefunction(parser.parse):
```

```
            return await parser.parse(source)
```

```
        # else run in thread to avoid blocking
```

```
        return await loop.run_in_executor(self._executor, functools.partial(asyncio.run,
parser.parse(source)))
```

```
    async def ingest_batch(self, sources: Sequence[str]) -> List[ContentNode]:
```

```
        results: List[ContentNode] = []
```

```
        tasks = []
```

```

for s in sources:
    if not self._rate.consume():
        # backpressure strategy: sleep a tiny bit
        await asyncio.sleep(0.01)
        tasks.append(asyncio.create_task(self._parse_in_thread(s)))
parsed = await asyncio.gather(*tasks)
for p in parsed:
    results.extend(p)
return results

```

```

# -----

```

```

# Vectorization (simulated) + Semantic Search

```

```

# -----

```

```

Vector = List[float]

```

```

def mock_embed(text: str, dim: int = 64) -> Vector:

```

```

    """A deterministic pseudo-embedding for demonstration (not a real model)."""

```

```

    h = hashlib.sha256(text.encode()).digest()

```

```

    vec = [((b % 128) / 128.0) for b in h[:dim]]

```

```

    # normalize

```

```

    norm = math.sqrt(sum(x * x for x in vec)) or 1.0

```

```

    return [x / norm for x in vec]

```

```
class SimpleVectorIndex:
```

```
    """A tiny in-memory vector index for semantic retrieval with cosine similarity.
```

```
    Demonstrates knowledge graph / semantic search patterns.
```

```
    """
```

```
    def __init__(self):
```

```
        self._index: Dict[str, Tuple[Vector, ContentNode]] = {}
```

```
    def upsert(self, node: ContentNode) -> str:
```

```
        v = mock_embed(node.body)
```

```
        self._index[node.id] = (v, node)
```

```
        return node.id
```

```
    def query(self, text: str, top_k: int = 5) -> List[Tuple[float, ContentNode]]:
```

```
        qv = mock_embed(text)
```

```
    def score(v1: Vector, v2: Vector) -> float:
```

```
        return sum(a * b for a, b in zip(v1, v2))
```

```
        scored = []
```

```
        for v, node in self._index.values():
```

```
            scored.append((score(qv, v), node))
```

```
scored.sort(key=lambda x: x[0], reverse=True)

return scored[:top_k]
```

```
# -----
# Knowledge Graph: lightweight triple store
# -----
```

```
@dataclass
```

```
class Triple:
```

```
    subject: str
```

```
    relation: str
```

```
    object: str
```

```
class TripleStore:
```

```
    def __init__(self):
```

```
        self._triples: List[Triple] = []
```

```
    def add(self, s: str, r: str, o: str):
```

```
        self._triples.append(Triple(s, r, o))
```

```
    def query(self, subject: Optional[str] = None, relation: Optional[str] = None, obj: Optional[str]
= None) -> List[Triple]:
```

```
        return [t for t in self._triples if (subject is None or t.subject == subject) and (relation is None
or t.relation == relation) and (obj is None or t.object == obj)]
```

Content Pipeline Controller

```
class ContentController:
```

```
def __init__(self, index: SimpleVectorIndex, triple_store: TripleStore, tracer: Tracer):
```

```
self.index = index
```

```
self.triples = triple_store
```

```
self.tracer = tracer
```

```
self._store: Dict[str, ContentNode] = {}
```

```
async def upsert_nodes(self, nodes: Sequence[ContentNode]) -> List[str]:
```

```
ids: List[str] = []
```

```
with self.tracer.start_span("content.upsert_batch"):
```

```
for n in nodes:
```

```
# dedupe via fingerprint
```

```
fp = n.fingerprint()
```

```
existing = self._store.get(fp)
```

```
# Here we store by fingerprint to show advanced dedupe technique
```

if existing is None:

```
self._store[fp] = n
```

```
self.index.upsert(n)
```

```
# extract tags to triples (semantic enrichment)
```

```
for tag in n.tags:
```

```
self.triples.add(n.id, "hasTag", tag)
```

```
        ids.append(n.id)

    return ids
```

```
def semantic_search(self, query: str, top_k: int = 5) -> List[Tuple[float, ContentNode]]:
    return self.index.query(query, top_k)
```

```
# -----
```

```
# CLI & Utilities
```

```
# -----
```

```
def slugify(text: str) -> str:
    return hashlib.md5(text.encode()).hexdigest()[:8]
```

```
# Example: a robust YAML frontmatter extractor (advanced regex + edge-cases)
```

```
def extract_front_matter(text: str) -> Tuple[Dict[str, Any], str]:
```

```
    import re
```

```
    m = re.match(r"^---\n(?:.*?\n)---\n(?:.*)$", text, re.S)
```

```
    if not m:
```

```
        return {}, text
```

```
    try:
```

```
        fm_raw = m.group(1)
```

```

body = m.group(2)

fm = {}

for line in fm_raw.splitlines():

    if ":" in line:

        k, v = line.split(":", 1)

        fm[k.strip()] = v.strip()

return fm, body

except Exception:

    return {}, text

```

```
# -----
```

```
# Example Usage & Integration test (async main)
```

```
# -----
```

```

async def _demo_pipeline():

    tracer: Tracer = container.get("tracer")

    index = SimpleVectorIndex()

    triples = TripleStore()

    controller = ContentController(index, triples, tracer)

```

```
md = """
```

```

---\npriority: high\n---\n# How to Reverse a Linked List\n\nThis doc explains how to reverse a
linked list in-place and why it's O(n).\n\nCode sample:\n```py\ndef reverse(head):\n    prev =
None\n    cur = head\n    while cur:\n        nxt = cur.next\n        cur.next = prev\n        prev = cur\n    cur = nxt\n    return prev\n```\n """

```

```
html = "<html><body><h1>API: Create User</h1><p>Use POST /users to create a  
user.</p></body></html>"
```

```
ing = Ingestor(parser_name="markdown", rate_limit=TokenBucket(rate=50, capacity=10))  
nodes = await ing.ingest_batch([md, html])
```

```
ids = await controller.upsert_nodes(nodes)  
print("Upserted IDs:", ids)
```

```
# Semantic search demo
```

```
print("Search Results for 'reverse linked list':")  
results = controller.semantic_search("reverse linked list", top_k=3)  
for score, node in results:  
    print(f"score={score:.3f} title={node.title[:60]} id={node.id}")
```

```
# Show triples
```

```
print("Triples:", triples.query())
```

```
# -----
```

```
# Testing utilities (lightweight) - mimicking CI tests
```

```
# -----
```

```
def _unit_tests() -> None:
```

```
# deterministic unit tests

# 1. Parser registration

assert "markdown" in PARSERS

assert "simple-html" in PARSERS


# 2. Fingerprint stability

n = ContentNode(id="1", title="t", body="b", created_at=datetime.utcnow(),
updated_at=datetime.utcnow())

f1 = n.fingerprint()

f2 = n.fingerprint()

assert f1 == f2


# 3. TokenBucket

tb = TokenBucket(rate=1.0, capacity=2.0)

assert tb.consume()


# 4. Mock embed deterministic length

v = mock_embed("hello world", dim=16)

assert len(v) == 16


print("All local unit tests passed.")


# -----

# Advanced: runtime code generation & AST manipulation (rare skill)
```

```
# -----
```

```
def generate_validation_function(fields: Sequence[str]) -> Callable[[Dict[str, Any]], bool]:
```

```
    """Generate a runtime validation function using exec() and AST-like safety checks.
```

This pattern demonstrates meta-programming which may be used in custom linters or content validators.

```
    """
```

```
    src_lines = ["def _validate(obj):"]
```

```
    src_lines.append("    if not isinstance(obj, dict):")
```

```
    src_lines.append("        return False")
```

```
    for f in fields:
```

```
        src_lines.append(f"        if '{f}' not in obj:")
```

```
            src_lines.append("            return False")
```

```
    src_lines.append("    return True")
```

```
    src = "\n".join(src_lines)
```

```
    loc: Dict[str, Any] = { }
```

```
    exec(src, { }, loc)
```

```
    return loc["_validate"]
```

```
# -----
```

```
# Advanced: Descriptor + Metaclass example
```

```
# -----
```

```
class Typed:

    def __init__(self, name: str, type_: Type):

        self.name = name

        self.type_ = type_

    def __get__(self, instance, owner):

        if instance is None:

            return self

        return instance.__dict__.get(self.name)

    def __set__(self, instance, value):

        if not isinstance(value, self.type_):

            raise TypeError(f"Expected {self.type_} for {self.name}")

        instance.__dict__[self.name] = value
```

```
class ModelMeta(type):

    def __new__(mcls, name, bases, namespace):

        annotations = namespace.get('__annotations__', {})

        for k, t in annotations.items():

            namespace[k] = Typed(k, t)

        return super().__new__(mcls, name, bases, namespace)
```

```
class SimpleModel(metaclass=ModelMeta):
```

id: str

count: int

Packaging-friendly entrypoint

def main(argv: Optional[List[str]] = None) -> int:

argv = argv or sys.argv[1:]

if "--test" in argv:

_unit_tests()

return 0

loop = asyncio.get_event_loop()

loop.run_until_complete(_demo_pipeline())

return 0

If executed as script, run demo

if __name__ == "__main__":

raise SystemExit(main())